

RÉSUMÉ THÉORIQUE – FILIÈRE DÉVELOPPEMENT WEB FULL STACK

Compétence M13 : « Développer en back-end »

120 heures

SOMMAIRE

1. Découvrir le Framework PHP Laravel

Découvrir les notions fondamentales des Frameworks PHP
Préparer l'environnement de Laravel

2. Programmer avec Laravel

Installation des outils pour Laravel
Créer un nouveau projet Laravel
Architecture d'un projet Laravel
Laravel Artisan
Lancer un projet Laravel

3. Approfondir la programmation Laravel

Gérer la sécurité
Interagir avec la base de données
Manipuler l'ORM Eloquent

4. Administrer un site à l'aide d'un CMS

Manipuler les éléments essentiels d'un CMS
Personnaliser graphiquement un site à l'aide d'un CMS
Manipuler les outils avancés d'un CMS





PARTIE 1

Découvrir le Framework PHP Laravel

Dans ce module, vous allez :

- Découvrir les notions fondamentales des Frameworks PHP
- Préparer l'environnement de Laravel

20 heures



CHAPITRE 1

Découvrir les notions fondamentales des Frameworks PHP

Ce que vous allez apprendre dans ce chapitre :

1. **Présentation des Frameworks PHP**
2. L’architecture MVC
3. Intérêt du Framework Laravel

02 heures

Qu'est-ce qu'un Framework web ?

Un Framework est **une boite à outils** pour **aider** les développeurs dans la réalisation de leurs tâches**. Frame (*cadre*) et work (*travail).***
Un Framework contient des composants autonomes qui permettent de **résoudre les problèmes souvent rencontrés par les développeurs** (CRUD, arborescence, normes, sécurités, etc.).
Un Framework n'est pas uniquement une boite à outils. Il peut aussi désigner une **méthodologie**.

Pourquoi utiliser un framework ?

L'objectif de l'utilisation d'un Framework est de :

- Maximiser la productivité du développeur qui l'utilise.
- Faciliter **la maintenance** du l'application Web réalisée.

Voici les trois raisons pour utiliser un Framework :

- **Rapidité :** le Framework permet un gain de temps et une livraison beaucoup plus rapide qu'un développement à zéro.
- **Organisation :** l'architecture d'un Framework favorise une bonne organisation du code source (modèle MVC par exemple).
- **Maintenabilité :** l'organisation du Framework facilite la maintenance du logiciel et la gestion des évolutions.

Copyright - Tout droit réservé - OFPPT

Les critères de choix d'un Framework PHP

1. La courbe d'apprentissage du framework ne devrait pas être trop dure.
2. Le framework PHP doit répondre aux **exigences techniques** de votre projet. (version PHP minimale , supporter la ou les bases de données du projet, …)
3. Un bon équilibre de **fonctionnalités du framework vis-à-vis du projet**. Choisir un framework minimal pour un petit projet.
4. Une **bonne documentation et un bon support** sont importants pour que vous puissiez tirer le meilleur parti de votre framework PHP.

Les Frameworks PHP les plus populaires

Il est difficile d'obtenir une liste définitive des frameworks PHP. Voici quelques-uns des meilleurs frameworks PHP en usage aujourd'hui :

1. Laravel
2. Symfony
3. Codeigniter
4. Zend Framework / Laminas Project
5. Yii Framework
6. …

Copyright - Tout droit réservé - OFPPT

Le framework Symphony

Symfony est un excellent choix pour les sites web et les applications qui doivent être évolutifs, il a plusieurs fonctionnalités:

- Supporte **la plupart des bases de données.**
- A son propre ORM **Doctrine.**
- Utilise le moteur de templating **Twig**, qui est facile à apprendre, rapide et sûr.
- **Packagist *** liste plus de **4 000 paquets Symfony** téléchargeable prête à utiliser.
- dispose d'un **support professionnel (**support comercial de Sensio Labs) , contrairement à la plupart des autres frameworks PHP.
- S'intègre facilement avec des frameworks populaires comme Drupal.



* Packagist est une sorte de répertoire public de packages PHP utilisables via Composer.
Composer est un gestionnaire de dépendances entre applications et librairies.
Composer permet de gérer pour chaque projet, la liste des modules et bibliothèques nécessaires à son fonctionnement ainsi que leurs versions. Il est utilisable via la console en ligne de commande. De plus, il permet de mettre en place un système d'autoload pour les bibliothèques compatibles.

Copyright - Tout droit réservé - OFPPT

Le framework Laravel

Laravel est présenté comme **« Le framework PHP pour les artisans du web ».**

Lancé en 2011, Laravel se trouve en tête des classements grâce à ses fonctionnalités:

- Offre une solution MVC complète**.**
- a son propre ORM **Eloquent ORM .**
- Utilise le moteur de templating **Blade.** On peut utiliser PHP dans Blade, contrairement aux autres.
- **Packalyst**, une collection de paquets Laravel, compte plus de 15 000 paquets utilisable .
- Dispose de l'outil de ligne de commande **Artisan Console** qui permet d'automatiser les tâches répétitives et de générer rapidement du code squelette.
- Dispose de plusieurs outils utiles tels que **Mix** pour la compilation des actifs CSS et JS, et **Socialite** pour l'authentification OAuth.
- Bénéficie d'une large communauté de développeurs



Copyright - Tout droit réservé - OFPPT

Le Zend framework/ Laminas Project

Zend Framework est un framework PHP établi de longue date qui est maintenant en transition vers Laminas Project.

La migration vers Laminas est fortement recommandée, car Zend n'est plus mis à jour.

- C'est un framework complètement orienté objet.
- Basé sur une méthodologie agile.
- Il suit strictement le modèle de conception MVC
- La communauté de Laminas dispose d'un forum et d'un groupe Slack pour la collaboration et le support.
- Le seul inconvénient de Zend réside dans le fait qu'il n'est pas aussi facile à maîtriser.

Copyright - Tout droit réservé - OFPPT

Le Yii Framework

Le nom de ce framework, **Yii**, signifie « simple et évolutif » en chinois. Il signifie également **« Yes, It Is ! »**.

Les principaux avantages de l'utilisation du framework Yii sont:

- C'est assez facile à installer et à démarrer
- Il est préchargé avec un modèle Bootstrap
- Il est livré avec un design élégant pour commencer rapidement
- C'est un framework PHP qui est également livré avec un débogueur
- Il est livré avec un générateur de code de classe extrêmement puissant appelé **Gii**. Le **générateur de code Gii** peut rapidement construire un squelette de code pour vous, ce qui permet de gagner du temps.
- La communauté Yii offre un support en direct via Slack ou IRC.

Copyright - Tout droit réservé - OFPPT

Ce que vous devez savoir avant d'utiliser un framework PHP

Avant d'utiliser un framework PHP, il faut connaitre :

- Le **langage PHP** lui-même. La plupart des frameworks fonctionnent avec PHP version 7.2 ou supérieure.
- Le PHP **orienté objet**, car la plupart des frameworks PHP modernes sont orientés objet.
- **Les bases de données et la syntaxe SQL.** Chaque framework PHP a sa propre liste de bases de données supportées.
- **ORM** (Object-Relational-Mapping) est une méthode d'accès aux données d'une base de données utilisant une syntaxe orientée objet au lieu d'utiliser le langage SQL. De nombreux frameworks PHP ont leur propre ORM intégré.
- Le fonctionnent **les serveurs web** comme Apache et Nginx.
- Architecture **MVC** (Modèle – Vue – Controleur). Les frameworks PHP suivent généralement le modèle de conception MVC
- L'utilisation d'une **interface en ligne de commande** **Laravel Artisan** (commande-line interface ou CLI) permet de se sentir à l'aise dans un framework PHP.

Copyright - Tout droit réservé - OFPPT



CHAPITRE 1

Découvrir les notions fondamentales des Frameworks PHP

Ce que vous allez apprendre dans ce chapitre :

1. Présentation des Frameworks PHP
2. **L'architecture MVC**
3. Intérêt du Framework Laravel

02 heures

L'architecture MVC

Les frameworks PHP suivent généralement le plus célèbres *design patterns* qui s'appelle MVC, qui signifie **Modèle - Vue - Contrôleur**.

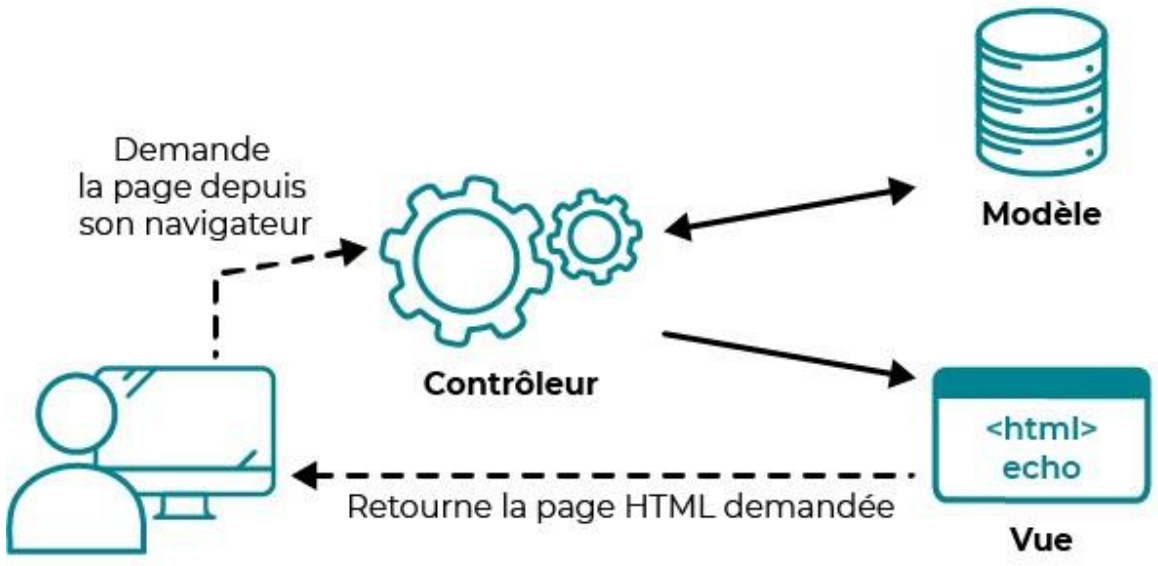
Le but du pattern MVC est de séparer la logique du code en trois parties que l'on retrouve dans des fichiers distincts.

- **Modèle** : cette partie gère la **logique métier**. Le modèle communique avec la BDD. C'est lui qui s'occupe de récupérer un article en base par exemple. On y trouve des algorithmes complexes et des requêtes SQL.
- **Vue** : cette partie se concentre sur l'**affichage**. Elle affiche à l'utilisateur des données contenues dans des variables. On y trouve essentiellement du code HTML mais aussi quelques boucles et conditions PHP très simples.
- **Contrôleur** : cette partie gère les **échanges** avec l'utilisateur. C'est en quelque sorte l'intermédiaire entre l'utilisateur, le modèle et la vue. Le modèle lui transmet les données récupérées en base de données, puis il transmet ces données à la vue.

Laravel propose le modèle MVC mais ne l'impose pas.

Copyright - Tout droit réservé - OFPPT

L'architecture MVC



Le client et l'architecture MVC

1. Le contrôleur reçoit des requêtes de l'utilisateur.
2. Le contrôleur va demander au modèle d'effectuer certaines actions (lire des informations depuis une base de données,…) et de lui renvoyer les résultats.
3. Le contrôleur va *adapter* ce résultat et le donner à la vue.
4. Enfin, le contrôleur va renvoyer la nouvelle page HTML, générée par la vue, à l'utilisateur.

Copyright - Tout droit réservé - OFPPT



CHAPITRE 1

Découvrir les notions fondamentales des Frameworks PHP

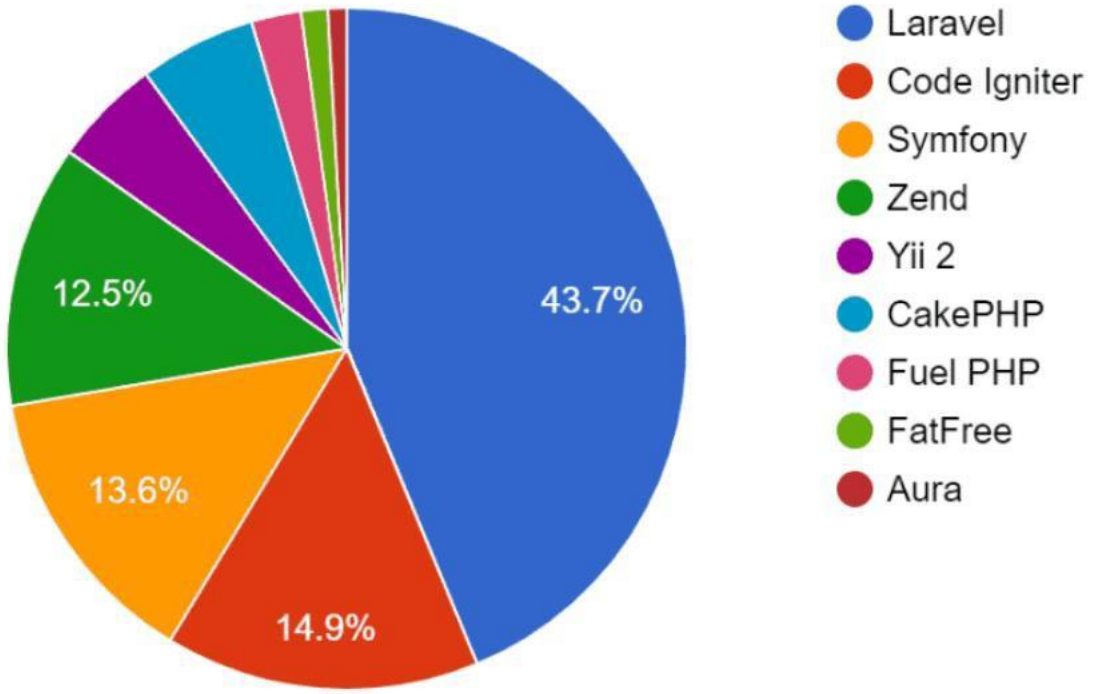
Ce que vous allez apprendre dans ce chapitre :

1. Présentation des Frameworks PHP
2. L'architecture MVC
3. **Intérêt du Framework Laravel**

02 heures

Pourquoi choisir le Framework Laravel ?

Aujourd'hui, Laravel est le framework PHP le plus utilisé car il peut gérer en toute sécurité des applications web complexes plus rapidement que d'autres frameworks.



https://laravel.sillo.org/cours-laravel-8-les-bases-presentation-generale/

PARTIE 1

Copyright - Tout droit réservé - OFPPT

Pourquoi choisir le Framework Laravel ?

1. **Sécurité de haut niveau**: Avec Laravel, votre application Web ne présente aucun risque d'injections SQL involontaires et cachées.
2. **Performance:** Laravel propose divers outils qui aident les développeurs à améliorer leurs performances.
3. **Bibliothèques orientées objet:** Laravel possède des bibliothèques orientées objet et d'autre pré installées, qui ne se trouvent dans aucun autre framework PHP. Comme la bibliothèque d'authentification.
4. **Documentation et communauté:** Laravel possède une puissante communauté de développeurs qui fournit en une assistance en permanence.
5. **Tests unitaires:** Avec les tests unitaires de Laravel, chaque module de votre application Web est testé avant la mise en ligne du site.
6. **Intégration aux services de messagerie :** Il fournit également des pilotes pour Mailgun, SMTP, Mandrill, SparkPost, la fonction de courrier électronique de PHP et Amazon SES.
7. **Créateur d'applications multilingues:** Le framework Laravel vous aide donc à créer facilement et rapidement votre application Web dans différentes langues.
8. **Tutoriels Laracasts :** Laravel propose des fonctionnalités de Laracasts, un mélange de tutoriels vidéo gratuits et payants qui vous montrent comment utiliser Laravel pour le développement.

Copyright - Tout droit réservé - OFPPT

Résumé

- Un framework fait gagner du temps et donne l'assurance de disposer de composants bien codés et fiables.
- Laravel est un framework novateur, complet, qui utilise les possibilités les plus récentes de PHP et qui est impeccablement codé et organisé.
- La documentation de Laravel est complète, précise et de plus en plus de tutoriels et exemples apparaissent sur la toile.
- Laravel adopte le patron MVC.
- Laravel est totalement orienté objet.

Copyright - Tout droit réservé - OFPPT



CHAPITRE 2

Préparer l’environnement de Laravel

1. **Installation des outils pour Laravel**
2. Créer un nouveau projet Laravel
3. Architecture d’un projet Laravel
4. Laravel Artisan
5. Lancer le projet Laravel

Les outils pour Laravel

Pour travailler avec Laravel, vous avez besoin de :

1. PHP
2. un serveur Local (Apache ou Nginx), permettant l'execution du PHP
3. un serveur de base de données pour gérer vos bases de données.
4. Composer, un gestionnaire de dépendances pour PHP
5. IDE et Editeurs de code (Visual Studio Code, Atom, PHPStorm …)
6. Navigateurs Web : Laravel est compatible sur tout les navigateurs récents (Chrome, Firefox ou Safari)

Nous vous recommandons d'installer les dernières versions de ces logiciels ayant une version PHP plus récente

(PHP 8.2 pour Laravel 11)

Documentation officielle **:https://laravel.com/docs/11.x**

Copyright - Tout droit réservé - OFPPT

Téléchargement et installation de PHP

- Pour installer PHP vous allez devoir vous rendre sur la page de téléchargement Windows du site officiel : http://windows.php.net/download/.
- Installer la version 8.2
- Vérifier que php est bien installé en exécutant la commande dans la console **php --version** ou **php -v**



Configuration des extensions nécessaires pour PHP

Les extensions **PDO, Tokenizer, OpenSSL et Mbstring** de PHP doivent être activées.

Dans le fichier de configuration de PHP **php.ini** , décommenter les lignes ci-dessous en enlevant le point virgule au début des lignes :

```
;extension=php_mbstring.dll
;extension=php_openssl.dll
```


Copyright - Tout droit réservé - OFPPT

Composer - Définition

Composer *est un logiciel gestionnaire de dépendances libre écrit en PHP. Il permet à ses utilisateurs de déclarer et d'installer les bibliothèques dont le projet principal a besoin.* Il permet de télécharger et de mettre à jour des bibliothèques externes.

Les bibliothèques externes permettent de réutiliser le code écrit par d'autres personnes pour simplifier le développement.

Exemple:

- Pour gérer des dates, vous pouvez utiliser **Carbon**
- Pour gérer les paiements Paypal, vous pouvez utiliser la bibliothèque officielle **PayPal PHP SDK**).

Composer permet également de créer des projets Laravel et de télécharger le framework.

Le framework Laravel est un simple assemblage de plusieurs dizaines de bibliothèques.



Copyright - Tout droit réservé - OFPPT

Installation de Composer

Pour installer Composer, il suffit de télécharger un installeur https://getcomposer.org/download/ et téléchargez **Composer-Setup.exe**.

Vérifiez lors de l'installation que le chemin par défaut vers PHP est bien **C:\PHP\php.exe.** car Composer est un fichier PHP et a besoin d'être exécuté.

Vérifications

Pour vérifier que tout fonctionne exécuter composer sur la ligne de commande comme suit:



Ici on peut voir que j'ai la version 2.8.4 de composer installée.

Copyright - Tout droit réservé - OFPPT



CHAPITRE 2

Préparer l’environnement de Laravel

1. Installation des outils pour Laravel
2. Créer un nouveau projet Laravel
3. Architecture d’un projet Laravel
4. Laravel Artisan
5. Lancer le projet Laravel

Utiliser l'installeur de Laravel

- **Installer Laravel dans notre ordinateur** : Ouvrez votre terminal (invite de commande) et tapez la ligne suivante: **composer global require laravel/installer**

```
C:\Users\hp
λ composer global require laravel/installer
```

require: ajoute la bibliothèque en paramètre au fichier *composer.json* et l'installe.

Créer notre premier projet laravel (laravelproject1) – Méthode 1 :

Taper la commande ***laravel new laravelproject1*** en plaçant le terminal dans le répertoire de travail (**ex: www ou htdocs**):

```
C:\
λ cd xampp\htdocs\

C:\xampp\htdocs
λ laravel new laravelproject1
```

PARTIE 1

Copyright - Tout droit réservé - OFPPT

Créer un nouveau projet Laravel avec Composer – Méthode 2:

Créer un deuxième projet à la racine du serveur nommé laravelproject2 :avec la commande suivante:

- Dirigez-vous vers votre dossier à la racine de votre serveur www ou htdocs et exécuter la commande : **composer create-project laravel/laravel laravelproject2**

```
C:\
λ cd xampp\htdocs\

C:\xampp\htdocs
λ composer create-project laravel/laravel laravelproject2
```

Ici nous avons demandé à composer de créer une installation Laravel dans le dossier «htdocs». Cette commande installera automatiquement la dernière version stable de Laravel.

Copyright - Tout droit réservé - OFPPT



CHAPITRE 2

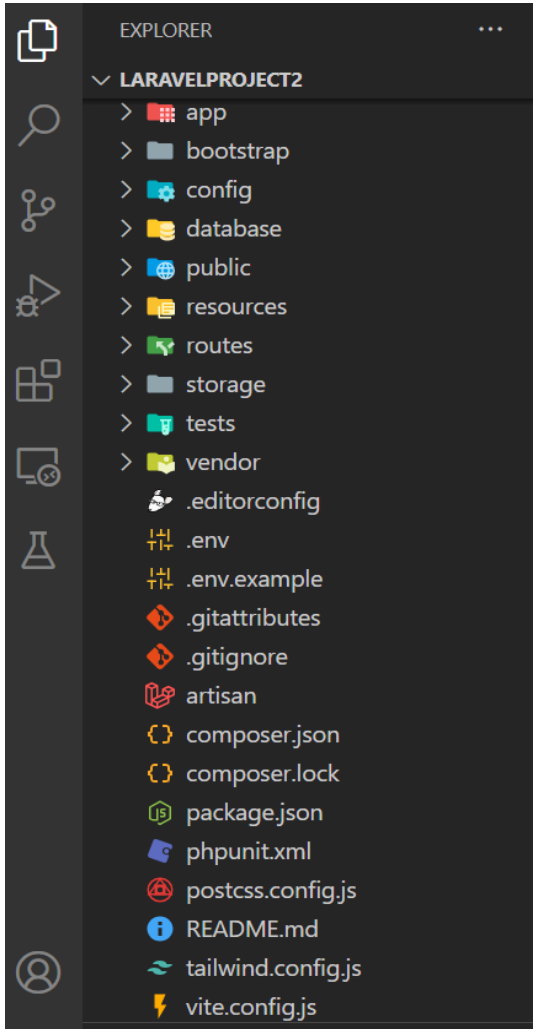
Préparer l’environnement de Laravel

1. Installation des outils pour Laravel
2. Créer un nouveau projet Laravel
3. Architecture d’un projet Laravel
4. Laravel Artisan
5. Lancer le projet Laravel

Architecture d'un projet Laravel

Ouvrez le dossier laravelproject2 avec l'éditeur Visual Code ou autre ,

vous devez avoir ceci comme résultat :

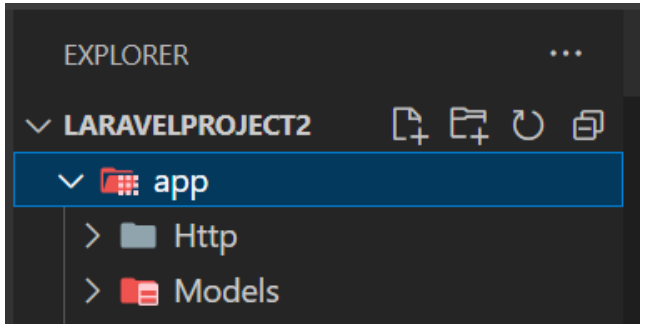


Copyright - Tout droit réservé - OFPPT

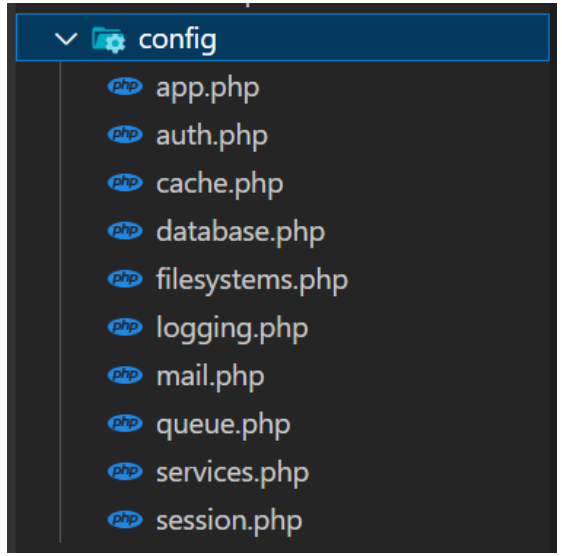
02 - Préparer l'environnement de Laravel

Architecture d'un projet Laravel

Les répertoires racines d'un projet Laravel

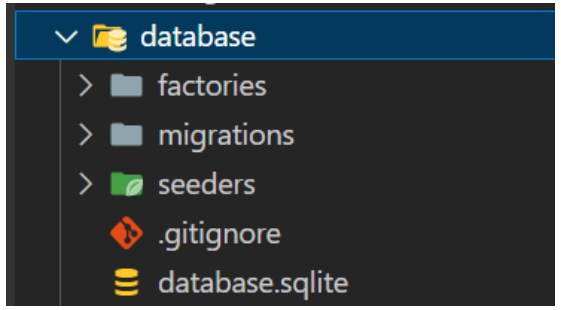


- **/app :** contient le code principal de votre application. Il contient deux répertoires principal http et Models.
  - **Le répertoire Http :**contient les contrôleurs et les middleware.
  - **Le répertoire Models :** contient toutes les classes de modèle Eloquent.



- **/config :** contient tous les fichiers de configuration de votre application, authentification, namespace, mails, base de données etc…

- **/database :** contient vos migrations de base de données, les données factices (faux data) de vos modèles.

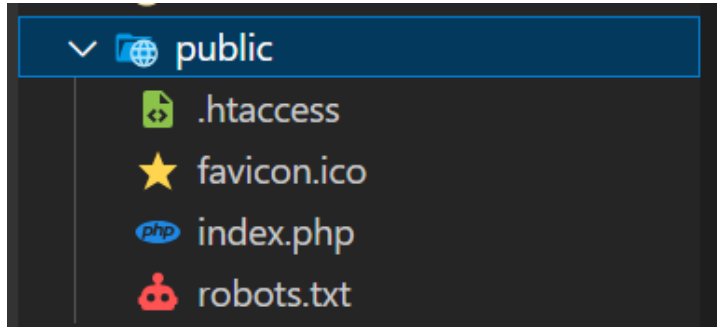


PARTIE 1

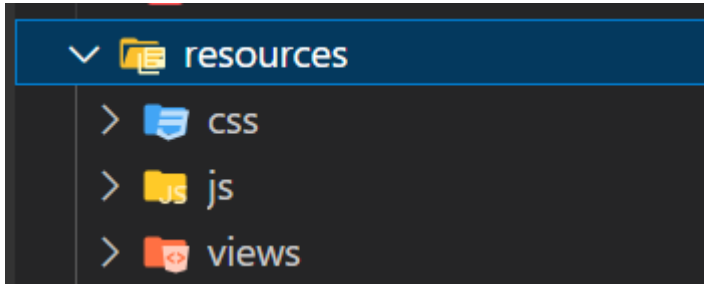
Copyright - Tout droit réservé - OFPPT

Les répertoires racines d'un projet Laravel

- **/public:** par convention comme pour la majorité des frameworks il s'agit du seul dossier accessible depuis le serveur où les fichiers sont accessibles depuis votre site (images, feuilles de style et scripts principalement).
  Contient le fichier index.php, qui est le point d'entrée pour toutes les demandes entrant dans votre application et configure le chargement automatique.



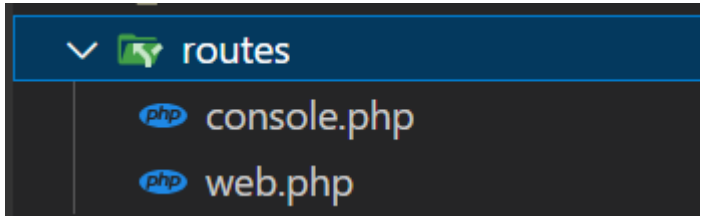
- **/resources:** contient vos **vues** ainsi que vos fichiers bruts non compilés tels que CSS, SASS ou JavaScript



- **/routes :**contient toutes les définitions des URLs pour votre application.

  2 fichiers de route sont inclus avec Laravel : web.php et console.php

  Le fichier web.php: utilisé pour définir les routes web de l'application. Ces routes sont généralement destinées à gérer les requêtes HTTP qui retournent des réponses HTML pour les utilisateurs finaux.



PARTIE 1

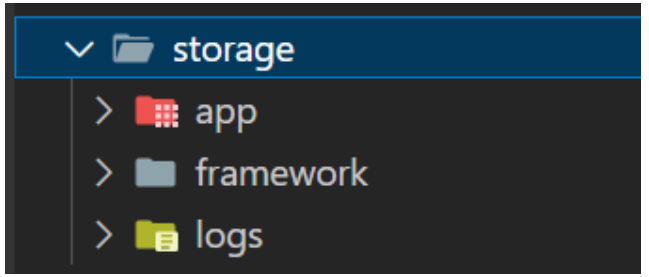
Copyright - Tout droit réservé - OFPPT

composer.json

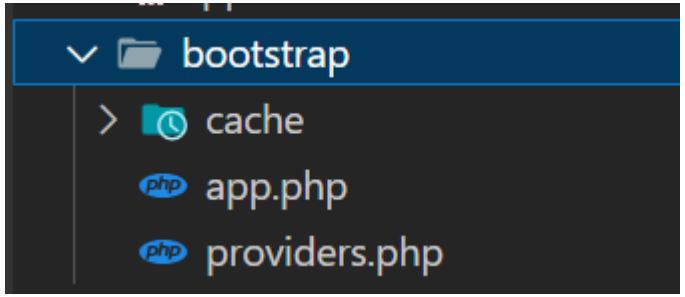
- **/storage:** contient vos logs, modèles de vues (blades) compilés, sessions basées sur des fichiers, fichiers caches et autres fichiers générés par le framework.

  Ce répertoire est divisé en répertoires app, framework et logs.

  - **storage/app** peut être utilisé pour stocker tous les fichiers générés par votre application.
  - **storage/framework** est utilisé pour stocker les fichiers et les caches générés par le framework.
  - **storage/logs** contient les fichiers journaux de votre application.
  - **storage/app/public:** peut être utilisé pour stocker des fichiers générés par l'utilisateur, tels que des avatars de profil, les images des vos publications, qui doivent être accessibles au public.
    - Vous devez créer un lien symbolique(raccourcis) public/storage pointant vers ce répertoire.
    - Vous pouvez créer le lien à l'aide de la commande Artisan : php artisan storage:link



- **/bootstrap** : démarrage de l'app.

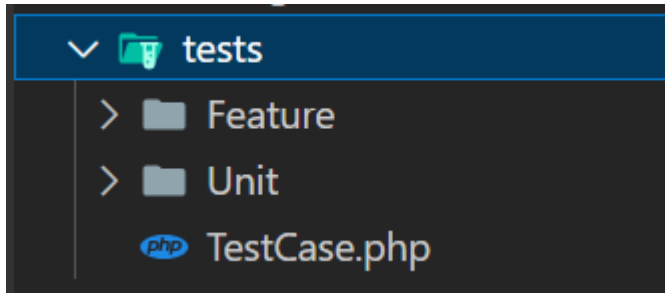


PARTIE 1

Copyright - Tout droit réservé - OFPPT

Les répertoires racines d'un projet Laravel

- **/tests:** contient vos tests automatisés. Des exemples de tests unitaires PHPUnit et de tests de fonctionnalités sont fournis prêts à l'emploi.
  - Chaque classe de test doit être suffixée par le mot **Test**.
  - Vous pouvez exécuter vos tests à l'aide des commandes php unit ou php vendor/bin/phpunit.
  - Ou, si vous souhaitez une représentation plus détaillée et plus belle de vos résultats de test, vous pouvez exécuter vos tests à l'aide de la commande Artisan php artisan test.



Copyright - Tout droit réservé - OFPPT

PARTIE 1

Qu'est ce que le fichier composer.json ?

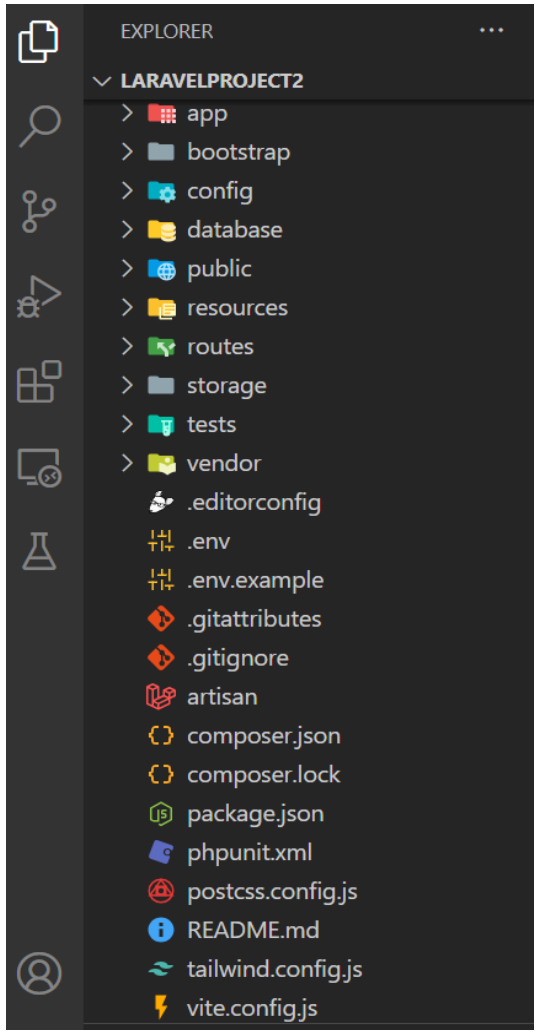
C'est le fichier qui stocke tous les paquets (dépendances) d'un projet, ainsi que leurs versions, mais peut aussi être utilisé pour définir un projet quand il est publié sur Git, par exemple.

composer.json

composer.json > {} scripts

Install | Update | Check

```
1   {
2       "$schema": "https://getcomposer.org/schema.json",
3       "name": "laravel/laravel",
4       "type": "project",
5       "description": "The Laravel framework.",
6       "keywords": ["laravel", "framework"],
7       "license": "MIT",
8       "require": {
9           "php": "^8.2",
10          "laravel/framework": "^11.31" (v11.35.1),
11          "laravel/tinker": "^2.9" (v2.10.0)
12      },
13  >   "require-dev": {···
21      },
22  >   "autoload": {···
```



Copyright - Tout droit réservé - OFPPT



CHAPITRE 2

Préparer l’environnement de Laravel

1. Installation des outils pour Laravel
2. Créer un nouveau projet Laravel
3. Architecture d’un projet Laravel
4. **Laravel Artisan**
5. Lancer le projet Laravel

Qu'est ce que Laravel Artisan?

Laravel Artisan est une Interface en Ligne de Commande (CLI) qui va vous permettre de gérer votre application en lançant des commandes via le terminal (effacer le cache de l'application, gérer des modèles, des contrôleurs, des routes…)

Laravel Artisan: commandes utiles

1. **Afficher la liste des commandes artisan:** php artisan list

```
php artisan list
```

2. **Obtenir de l'aide pour une commande spécifique:** php artisan help <commande>

```
php artisan help migrate
```

3. **Effacer le cache de l'application:** php artisan cache:clear

```
php artisan cache:clear
```

4. **Vérifier la version de Laravel installée:** php artisan --version ou bien php artisan -V

```
php artisan --version
```

```
php artisan -V
```


Copyright - Tout droit réservé - OFPPT

Laravel Artisan: commandes utiles (suite)

5. Lancer un serveur de développement local : php artisan serve

```
php artisan serve
```

6. Créer un contrôleur: php artisan make:controller NomDuController

```
php artisan make:controller IndexController
```

7. Créer un modèle: php artisan make:model NomDuModel

```
php artisan make:model UserModel
```

8. Créer un middleware: php artisan make:middleware NomDuMiddleware

```
php artisan make:middleware CheckAge
```

9. Afficher les routes enregistrées : php artisan route:list

```
php artisan route:list
```

10. Exécuter toutes les migrations: php artisan migrate

```
php artisan migrate
```

11. Rollback (annuler la dernière migration) : php artisan migrate:rollback

```
php artisan migrate:rollback
```

12. Créer un modèle avec une migration associée : php artisan make:model NomDuModel -m

```
php artisan make:model UserModel -m
```

13. Réinitialiser et relancer toutes les migrations: php artisan migrate:refresh

```
php artisan migrate:refresh
```

14. Générer une clé d'application (stockée dans .env) : php artisan key:generate

```
php artisan key:generate
```


Copyright - Tout droit réservé - OFPPT



Résumé rapide des commandes essentielles :

Commande	Utilité
`php artisan serve`	Lancer le serveur local
`php artisan make:controller`	Créer un contrôleur
`php artisan make:model`	Créer un modèle
`php artisan migrate`	Exécuter les migrations
`php artisan route:list`	Voir les routes disponibles
`php artisan cache:clear`	Vider le cache
`php artisan test`	Lancer les tests
`php artisan key:generate`	Générer une clé d'application

Copyright - Tout droit réservé - OFPPT



CHAPITRE 2

Préparer l’environnement de Laravel

1. Installation des outils pour Laravel
2. Créer un nouveau projet Laravel
3. Architecture d’un projet Laravel
4. Laravel Artisan
5. **Lancer le projet Laravel**

Lancer le projet Laravel

1. Placez le terminal dans le dossier racine de votre projet et taper les commandes*: php artisan key:generate* ensuite *php artisan serve* :

```
C:\xampp\htdocs\laravelproject2
λ php artisan key:generate

  INFO  Application key set successfully.


C:\xampp\htdocs\laravelproject2
λ php artisan serve
  forking is not supported on this platform

  INFO  Server running on [http://127.0.0.1:8000].

  Press Ctrl+C to stop the server
```

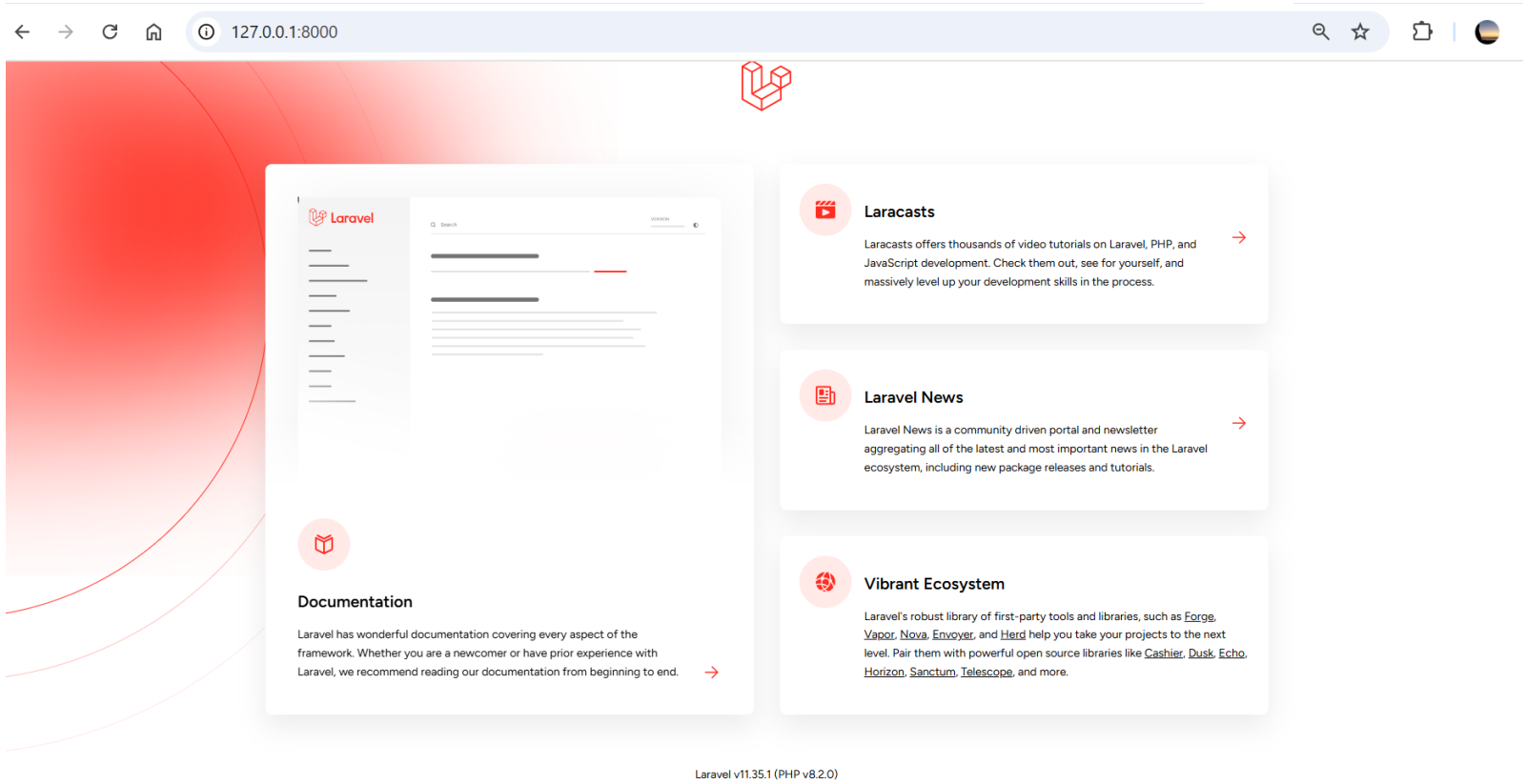
Sous windows 10 , en cas d'erreur exécuter la commande suivante : **composer install --ignore-platform-reqs**

*On peut utiliser **php artisan key:generate** dans le cas d'un clone d'un autre projet Laravel pré-crée (afin de regénérer les clés du projet)*

Copyright - Tout droit réservé - OFPPT

Lancer le projet Laravel

1. Ouvrez le navigateur et allez sur http://localhost/:8000, et vous obtiendrez le résultat suivant :



Copyright - Tout droit réservé - OFPPT

Résumé

- Pour son installation et sa mise à jour, Laravel utilise le gestionnaire de dépendances **Composer**.
- La création d'une application Laravel se fait à partir de la console avec une simple ligne de commande.
- Laravel est organisé en plusieurs dossiers.
- Le dossier public est le seul qui doive être accessible par le serveur.

Copyright - Tout droit réservé - OFPPT



PARTIE 1

Programmer avec Laravel

Dans ce module, vous allez :

- Connaître les fondements du modèle MVC Laravel
- Maîtriser le Framework Laravel

30 heures



CHAPITRE 1

Gestion du routage

Ce que vous allez apprendre dans ce chapitre :

- Routage de base (redirection, affichage, liste de routes)
- Paramètres de routage
- Routes nommées
- Groupe de routage
- Liaisons de modèles de routes
- Routes de repli
- (spoofing) de la méthode du formulaire
- Accès à la route courante
- Cross Origin Resource Sharing (CORS)
- Mise en cache des routes

05 heures



CHAPITRE 1

Gestion du routage

1. Routage de base (redirection, affichage, liste de routes)
2. Paramètres de routage
3. Routes nommées
4. Groupe de routage
5. Liaisons de modèles de routes
6. Routes de repli
7. (spoofing) de la méthode du formulaire
8. Accès à la route courante
9. Cross Origin Resource Sharing (CORS)
10. Mise en cache des routes

App\Http\Controllers

```php
<?php

namespace App\Http\Controllers;

use App\Service\ApiService;

class IndexController
{

public function index(ApiService $apiService)
{
    echo $apiService->number();
}

}
```

App\Service

```php
<?php

namespace App\Service;

class ApiService
{

public function number() {
    return '1';
}

}
```

App\Service\Request

```php
<?php

namespace App\Service\Request;

class ApiService
{
public function number() {
    return '2';
}

}
```


Copyright - Tout droit réservé - OFPPT

- Les routes **Laravel** les plus basiques acceptent un URI et une fermeture, fournissant une méthode très simple et expressive de définition des routes et du comportement sans fichiers de configuration de routage compliqués :

```
use Illuminate\Support\Facades\Route;

Route::get('/greeting', function () {
• return 'Hello World';
});
```


Copyright - Tout droit réservé - OFPPT

Les fichiers de routage par défaut

- Toutes les routes **Laravel** sont définies dans vos fichiers de route, qui se trouvent dans le répertoire **routes**.
- Ces fichiers sont automatiquement chargés par le fichier **App\Providers\RouteServiceProvider**.
- Le fichier **routes/web.php** définit les itinéraires qui sont destinés à votre interface Web.
- Ces routes sont affectées au webgroupe middleware, qui fournit des fonctionnalités telles que l'état de session et la protection CSRF. Les routes dans **routes/api.php** sont sans état et sont affectées au **api** groupe middleware.
- Pour la plupart des applications, vous commencerez par définir des routes dans votre fichier **routes/web.php**. Les itinéraires définis dans **routes/web.php** sont accessibles en saisissant l'URL de l'itinéraire défini dans votre navigateur. Par exemple, vous pouvez accéder à l'itinéraire suivant en naviguant vers **http://example.com/user** dans votre navigateur :

```
use Illuminate\Support\Facades\Route;

Route::get('/greeting', function () {
return 'Hello World';
});
```

- Les routes définies dans le fichier **routes/api.php** sont imbriquées dans un groupe de routes par le **RouteServiceProvider**. Dans ce groupe, le préfixe URI **/api** est automatiquement appliqué, vous n'avez donc pas besoin de l'appliquer manuellement à chaque route du fichier. Vous pouvez modifier le préfixe et d'autres options de groupe de routage en modifiant votre classe **RouteServiceProvider**.

Copyright - Tout droit réservé - OFPPT

Méthodes de routeur disponibles

- Le routeur vous permet d'enregistrer des routes qui répondent à n'importe quel verbe HTTP :

```
Route::get($uri, $callback);
Route::post($uri, $callback);
Route::put($uri, $callback);
Route::patch($uri, $callback);
Route::delete($uri, $callback);
Route::options($uri, $callback);
```

- Parfois, vous devrez peut-être enregistrer une route qui répond à plusieurs verbes HTTP. Vous pouvez le faire en utilisant la méthode **match**. Ou, vous pouvez même enregistrer une route qui répond à tous les verbes HTTP en utilisant la méthode any :

```
Route::match(['get', 'post'], '/', function () {
//
});

Route::any('/', function () {
//
});
```


Copyright - Tout droit réservé - OFPPT

Injection de dépendance

- Vous pouvez indiquer toutes les dépendances requises par votre itinéraire dans la signature de rappel de votre itinéraire. Les dépendances déclarées seront automatiquement résolues et injectées dans le rappel par le conteneur de service **Laravel** .
- Par exemple, vous pouvez donner un indice de type à la classe **Illuminate\Http\Request** pour que la requête HTTP actuelle soit automatiquement injectée dans votre rappel de route :

```
use Illuminate\Http\Request;

Route::get('/users', function (Request $request) {
// ...
});
```

Figure 1 : les navigateurs les plus utilisés

Copyright - Tout droit réservé - OFPPT

Redirection

- Si vous définissez une route qui redirige vers un autre **URI**, vous pouvez utiliser la méthode **Route::redirect**. Cette méthode fournit un raccourci pratique pour que vous n'ayez pas à définir une route ou un contrôleur complet pour effectuer une simple redirection :

```
Route::redirect('/ici', '/là-bas');
```

- Par défaut, **Route::redirect** renvoie un code d'état **302**. Vous pouvez personnaliser le code d'état à l'aide du troisième paramètre facultatif :

```
Route::redirect('/ici', '/là-bas', 301);
```

- Ou, vous pouvez utiliser la méthode **Route::permanentRedirect** pour renvoyer un code d'état **301**:

```
Route::permanentRedirect ('/ici', '/là-bas ');
```

*- Une redirection 301 est utilisée lorsque vous souhaitez rediriger de manière **permanente** une URL vers une autre URL. Cette redirection indique aux utilisateurs et aux moteurs de recherche qu'une page a été déplacée de façon permanente. Elle est le plus souvent utilisée lorsque des pages sont déplacées de façon permanente vers une autre URL ou ont été purement et simplement supprimées.*
À l'inverse, une redirection 302 est une redirection temporaire. Les redirections 302 sont utilisées lorsque le webmaster a l'intention de revenir à l'URL d'origine à l'avenir.

copyright - Tout droit réservé - OFPPT

Afficher les routes

- Si votre itinéraire ne doit renvoyer qu'une vue , vous pouvez utiliser la méthode **Route::view**.
- Comme la méthode **redirect**, cette méthode fournit un raccourci simple pour que vous n'ayez pas à définir une route ou un contrôleur complet. La méthode **view** accepte un URI comme premier argument et un nom de vue comme second argument. De plus, vous pouvez fournir un tableau de données à transmettre à la vue en tant que troisième argument facultatif :

```
Route::view('/welcome', 'welcome');

Route::view('/welcome', 'welcome', ['name' => ‘Ahmed']);
```

Source : https://www.wddonline.com/2016/09/27/history-world-wide-web/

Copyright - Tout droit réservé - OFPPT

La liste des routes

- La commande **Artisan route:list** peut facilement fournir une vue d'ensemble de toutes les routes définies par votre application :

```
php artisan route:list
```

- Par défaut, le middleware de route affecté à chaque route ne sera pas affiché dans la sortie **route:list** ; cependant, vous pouvez demander à **Laravel** d'afficher le middleware de route en ajoutant l' option -v à la commande :

```
php artisan route:list -v
```

- Vous pouvez également demander à **Laravel** de n'afficher que les routes commençant par un **URI** donné :

```
php artisan route:list --path=api
```

- De plus, vous pouvez demander à **Laravel** de masquer toutes les routes définies par des packages tiers en fournissant l'option **--except-vendor** lors de l'exécution de la commande **route:list** :

```
php artisan route:list --except-vendor
```


Copyright - Tout droit réservé - OFPPT

La liste des routes

- De même, vous pouvez également demander à **Laravel** de n'afficher que les routes définies par des packages tiers en fournissant l'option **--only-vendor** lors de l'exécution de la commande **route:list** :

```
php artisan route:list
```


Copyright - Tout droit réservé - OFPPT



CHAPITRE 1

Gestion du routage

1. Routage de base (redirection, affichage, liste de routes)
2. Paramètres de routage
3. Routes nommées
4. Groupe de routage
5. Liaisons de modèles de routes
6. Routes de repli
7. (spoofing) de la méthode du formulaire
8. Accès à la route courante
9. Cross Origin Resource Sharing (CORS)
10. Mise en cache des routes

Paramètres requis

- Parfois, vous devrez capturer des segments de l'URI dans votre route. Par exemple, vous devrez peut-être capturer l'ID d'un utilisateur à partir de l'URL. Vous pouvez le faire en définissant les paramètres de route :

```
Route::get('/user/{id}', function ($id) {
return 'User '.$id;
});
```

- Vous pouvez définir autant de paramètres de route que requis par votre route :

```
Route::get('/posts/{post}/comments/{comment}', function ($postId, $commentId) {
//
});
```

- Les paramètres de route sont toujours entourés d'accolades **{}** et doivent être composés de caractères alphabétiques. Les traits de soulignement **(_)** sont également acceptables dans les noms de paramètre de route. Les paramètres de route sont injectés dans les rappels/contrôleurs de route en fonction de leur ordre - les noms des arguments de rappel/contrôleur de route n'ont pas d'importance.

Source : https://www.wddonline.com/2016/09/27/history-world-wide-web/

Copyright - Tout droit réservé - OFPPT

Paramètres et injection de dépendance

- Si votre route a des dépendances que vous aimeriez que le conteneur de service **Laravel** injecte automatiquement dans le rappel de votre route, vous devez lister vos paramètres de route après vos dépendances.

```
use Illuminate\Http\Request;

Route::get('/user/{id}', function (Request $request, $id) {
return 'User '.$id;
});
```


Copyright - Tout droit réservé - OFPPT

Paramètres facultatifs

- Parfois, vous devrez peut-être spécifier un paramètre de route qui n'est pas toujours présent dans l'URI. Vous pouvez le faire en plaçant une **?** marque après le nom du paramètre. Assurez-vous de donner une valeur par défaut à la variable correspondante de la route :

```
Route::get('/user/{name?}', function ($name = null) {
return $name;
});

Route::get('/user/{name?}', function ($name = Ahmed') {
return $name;
});
```


Copyright - Tout droit réservé - OFPPT



CHAPITRE 1

Gestion du routage

1. Routage de base (redirection, affichage, liste de routes)
2. Paramètres de routage
3. Routes nommées
4. Groupe de routage
5. Liaisons de modèles de routes
6. Routes de repli
7. (spoofing) de la méthode du formulaire
8. Accès à la route courante
9. Cross Origin Resource Sharing (CORS)
10. Mise en cache des routes

- Les routes nommées permettent la génération pratique d'URL ou de redirections pour des routes spécifiques. Vous pouvez spécifier un nom pour une route en enchaînant la méthode **name** sur la définition de route :

```
Route::get('/user/profile', function () {
//
})->name('profile');
```

- Vous pouvez également spécifier des noms de route pour les actions du contrôleur :

```
Route::get(
'/user/profile',
[UserProfileController::class, 'show']
)->name('profile');
```


Copyright - Tout droit réservé - OFPPT

Génération d'URL vers des routes nommées

- Une fois que vous avez attribué un nom à une route donnée, vous pouvez utiliser le nom de la route lors de la génération d'URL ou de redirections via les fonctions de **Laravel** route et d'assistance **redirect** :

```
// Generating URLs...
$url = route('profile');

// Generating Redirects...
return redirect()->route('profile');
```

- Si la route nommée définit des paramètres, vous pouvez passer les paramètres comme deuxième argument à la fonction **route**. Les paramètres donnés seront automatiquement insérés dans **l'URL** générée dans leurs positions correctes :

```
Route::get('/user/{id}/profile', function ($id) {
//
})->name('profile');

$url = route('profile', ['id' => 1]);
```


Copyright - Tout droit réservé - OFPPT

- Si vous transmettez des paramètres supplémentaires dans le tableau, ces paires **clé/valeur** seront automatiquement ajoutées à la chaîne de requête de l'URL générée :

```
Route::get('/user/{id}/profile', function ($id) {
//
})->name('profile');

$url = route('profile', ['id' => 1, 'photos' => 'yes']);

// /user/1/profile?photos=yes
```


Copyright - Tout droit réservé - OFPPT